Universidad Mayor
Facultad de Ciencia - Ingeniería y Tecnología
Escuela de Ingeniería Civil

Propuesta de proyecto “Abarrato”

Equipo Abarrato

Carrera: Ingeniería Civil en Computación e Informática

Asignatura: Arquitectura de Desarrollo (Móvil y Web) | INGT1003-001

Profesor: Carlos Muñoz S.
Nombre(s) de estudiante(s):
- Lara Fuentes Lucas
- Lara Menares Valentina
Ciudad: Santiago
Fecha: 19/09/2026

Introducción: **1**
Desarrollo **2**
1.- Arquitectura de Información: 2
1.1.- Organización y Etiquetado: 2
1.2.- Navegación Móvil (Mobile-first): 2
1.3.- Flujo principal: 3
2.- Arquitectura Cliente-Servidor y Capas: 4
3.- Selección de Tecnologías y Frameworks: 6
4.- Decisión de Renderizado: 7
Conclusión: **8**
Bibliografía: **9**
Anexos: **10**

Introducción:

Durante los últimos años y hasta la actualidad, el precio de los alimentos básicos ha aumentado y se ha convertido en una de las principales preocupaciones para las familias chilenas. Según el Instituto Nacional de Estadísticas (2026), el Índice de Precios al Consumidor acumuló un alza de 40.7% aproximadamente entre diciembre de 2020 hasta agosto de 2026. A esto se le suma que la Canasta Básica de Alimentos alcanzó un valor de $89.570 por persona en enero de 2026 (Ministerio de Desarrollo Social y Familia, 2026). Frente a este incremento en el costo de vida básico, las personas buscan gastar la menor cantidad en abarrotes y esto presenta una problemática logística y económica: saber qué supermercados cerca de su ubicación poseen los productos más accesibles en comparación con los demás. La falta de conocimiento de este punto obliga a las personas a comprar productos a ciegas, perdiendo dinero al adquirir opciones más caras.

Para las familias o personas de pocos recursos que viven esta situación cada vez que llega la hora de adquirir insumos básicos, nuestro producto proporciona una plataforma móvil que permitirá consultar y comparar el valor exacto de los abarrotes en supermercados en un área determinada alrededor del consumidor. De esta forma, garantizamos que cada viaje de compras sea la opción más económica y eficiente posible.

La finalidad de este documento es presentar la propuesta comercial y técnica inicial de nuestra aplicación, demostrando la viabilidad de este proyecto para asegurar que la solución propuesta logra resolver de manera efectiva el problema planteado.

Desarrollo

1.- Arquitectura de Información:

La arquitectura de la información de “Abarrato” se diseñó estructurando las funciones para que los usuarios puedan encontrar de manera veloz lo que buscan y completen su objetivo de ahorrar dinero en sus compras diarias/semanales o mensuales.

1.1.- Organización y Etiquetado:

Se optó por un esquema de organización enfocado en las tareas del usuario, utilizando la jerarquía ancha y baja para evitar que la navegación se vuelva laberíntica. El etiquetado utiliza un lenguaje cotidiano neutro que permite a los usuarios comprender el significado de cada opción sin caer en confusión; por ejemplo, el apartado de despensa es empleado para el inventario personal y la configuración del algoritmo de cálculo tiene como nombre “¿Qué priorizas?”, evitando utilizar frases que el consumidor podría no comprender.

1.2.- Navegación Móvil (Mobile-first):

La aplicación concentra la navegación global en una barra inferior o “Tab Bar” al alcance del pulgar, lo que permite una fácil navegación en dispositivos móviles. El mapa del producto se divide en cuatro secciones principales:

- Inicio: Dashboard con el ahorro semanal y accesos directos hacia el “Modo Presupuesto” y “¿Qué puedo cocinar?”.

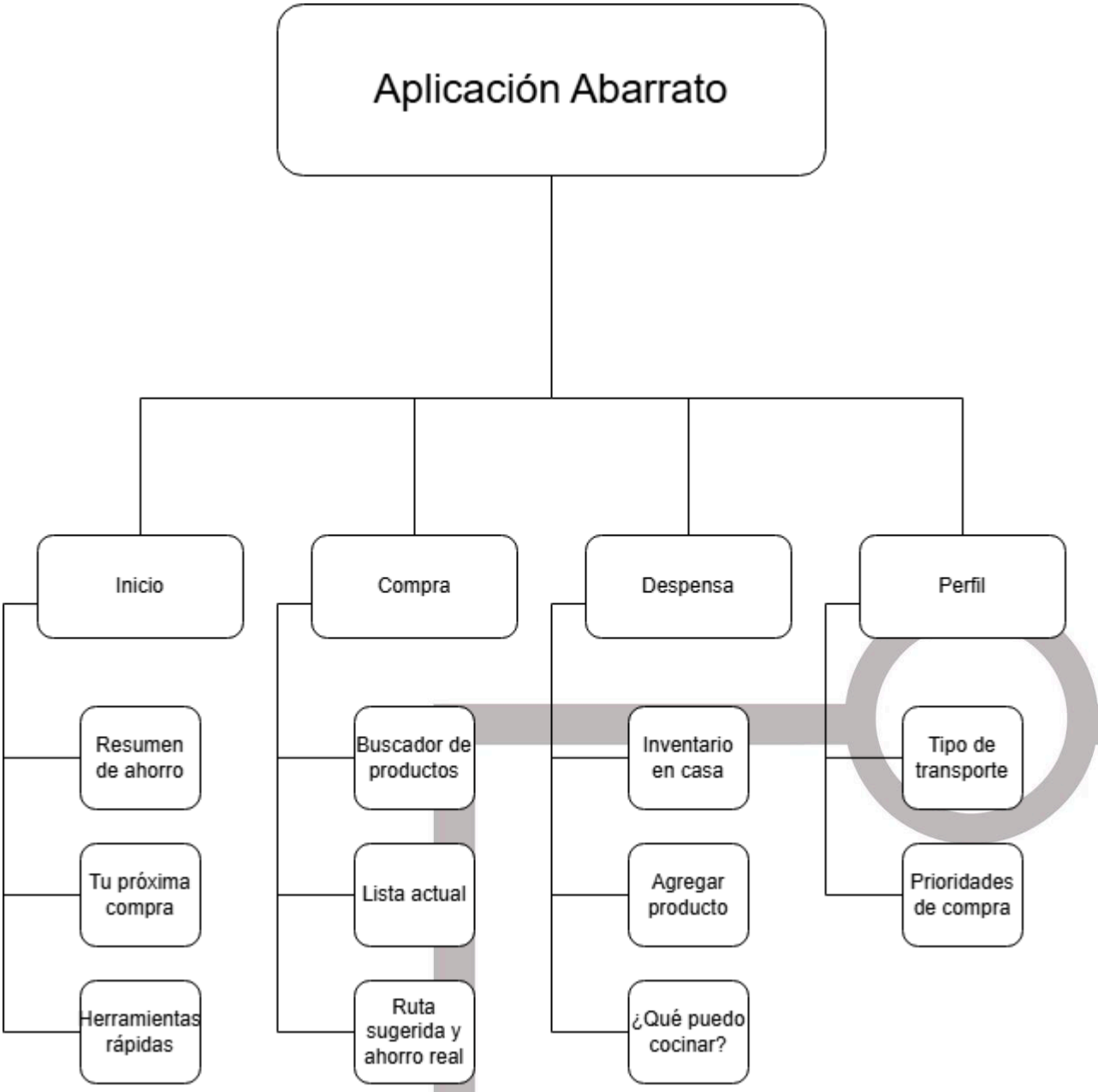
- Compra: Buscador de productos con filtros, en el que el usuario puede armar su lista y seleccionar la optimización que muestra los supermercados con mejores resultados según el tipo de ahorro.

- Despensa: Inventario personal del usuario que se actualiza manualmente, sirve para llevar la cuenta de los productos que faltan y se poseen.

- Perfil: Preferencias de transporte y tipos de ahorro que se acomodan a la necesidad principal del usuario.

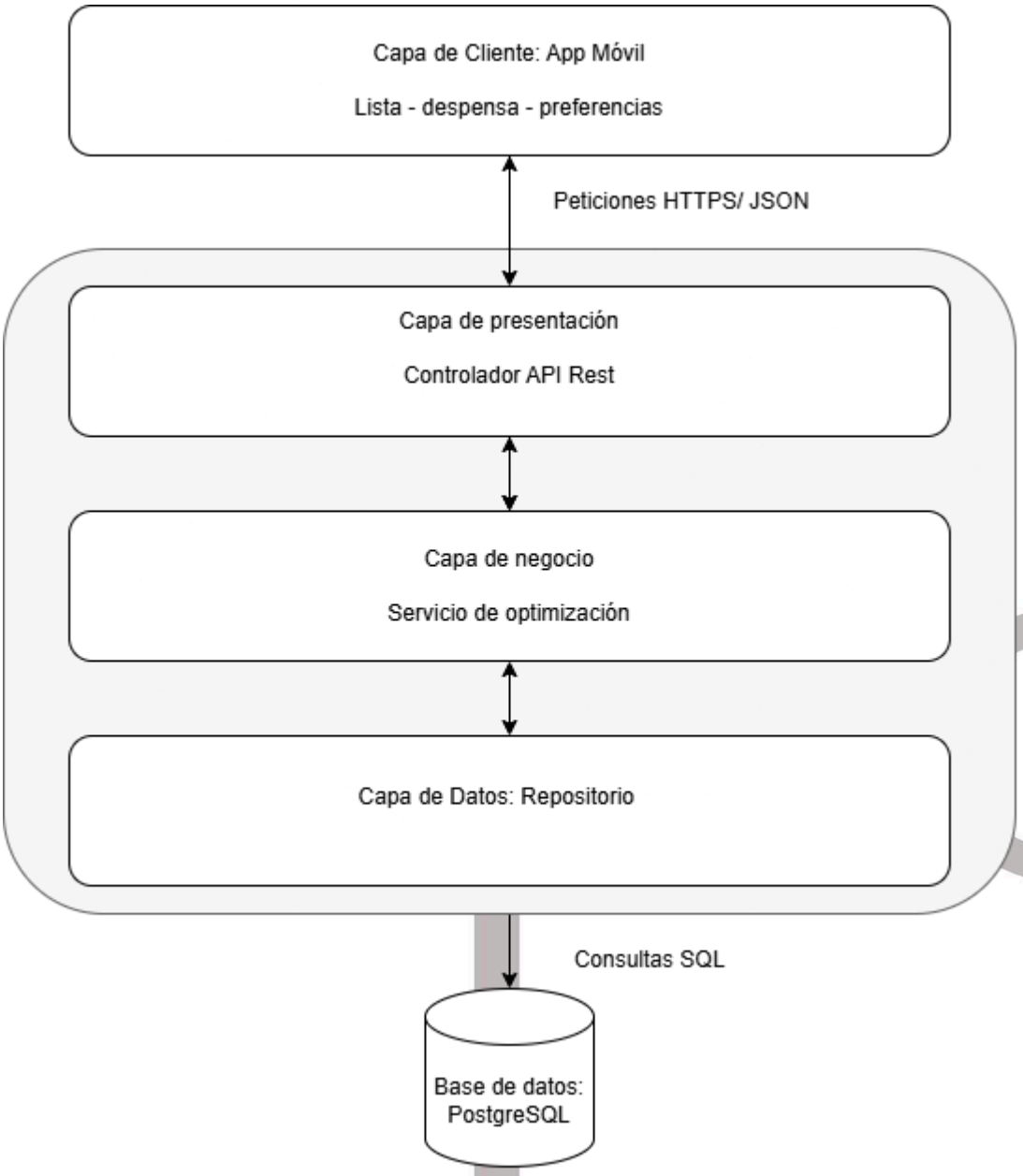
1.3.- Flujo principal:

El flujo de usuario está diseñado paso a paso para evitar confusiones o pérdida de rumbo:
[Inicio] -> [Crear nueva compra] -> [Buscador] -> [Optimizar] -> [Comparativa de ahorro y ruta] -> [Confirmar]



2.- Arquitectura Cliente-Servidor y Capas:

La plataforma trabaja bajo el modelo cliente-servidor, aislando las responsabilidades del backend en tres capas estrictas para asegurar escalabilidad, seguridad y facilitar el mantenimiento.



Para ilustrar esta separación, a continuación se detalla el trayecto de la petición cuando un usuario optimiza su lista de compras en un radio de dos kilómetros:

- Cliente (App Móvil Nativa en Java): El usuario solicita optimizar su carrito. La aplicación ensambla la petición con la lista y las coordenadas GPS precisas, enviándolas de forma segura a través de HTTPS al servidor.

- Capa de Presentación (Controlador API REST): Recibe la petición HTTP y funciona como primera parada. Valida únicamente la estructura de los datos entrantes.
- Capa de Negocio (Servidor): Ejecuta las reglas centrales. Recibe los datos validados, cruza la ubicación del usuario con los precios de los supermercados en el radio predefinido y calcula matemáticamente la ruta que proporciona la mayor cantidad de ahorro real, descontando los costos de transporte.
- Capa de Datos (Repositorio): Funciona como un puente hacia el almacenamiento. Traduce las solicitudes del servicio a consultas SQL parametrizadas para conseguir del catálogo los precios actualizados y el stock de los productos que fueron solicitados.
- Retorno: La base de datos (PostgreSQL) entrega la información a la Capa de Datos, que luego se sube a la Capa de Negocio para terminar con el cálculo. La capa de Presentación "formatea" este resultado en un documento JSON ligero y lo envía de vuelta a la aplicación móvil nativa para que se muestre en pantalla.

3.- Selección de Tecnologías y Frameworks:

Para la construcción de la aplicación "Abarrato" se evaluaron distintas tecnologías basándose en criterios de rendimiento móvil, facilidad de mantenimiento y robustez de procesamiento.

- Front-end (App Móvil) Nativo (Android/Java) vs. Multiplataforma (Flutter):
  Se evaluó el utilizar un framework multiplataforma como lo es Flutter para poder llegar a Android e IOS al mismo tiempo sin necesidad de utilizar dos códigos distintos, pero la decisión final se dejó en desarrollar una aplicación Nativa para Android usando Java en Android Studio. Esta elección se fundamentó en el hecho de que la falta de experiencia con otros lenguajes de programación, así como en la creación de aplicaciones móviles podría retrasar u obstaculizar el desarrollo del producto.
    - Desventaja: Al optar por un desarrollo nativo sólo para Android, se asume la desventaja comercial de dejar de lado por completo a los usuarios que utilizan los sistemas operativos iOS (iPhone). Llegar a ese público requerirá construir una segunda base de código especialmente centrada en ese SO o volver a construir desde cero, utilizando un framework multiplataforma para evitar tener dos sistemas de códigos.

- Back-end - Node.js vs. Java (Spring Boot):
  Se consideró Node.js en un principio por su velocidad inicial de desarrollo, pero finalmente se optó por Java Spring Boot. El procesamiento de combinaciones de

productos, precios y rutas necesita cálculos matemáticos intensivos donde el tipado fuerte y el manejo de concurrencia de Java ofrecen una robustez superior para evitar posibles fallos de memoria en el servidor bajo la carga de usuarios.

- Desventaja: Spring Boot tiene un mayor consumo de memoria RAM base y un tiempo de inicio de carga más lento que los scripts que utiliza Node.js, lo que exigirá proporcionar a los servidores con las características específicas necesarias para que nuestra infraestructura funcione.

4.- Decisión de Renderizado:

Debido a la naturaleza del modelo de negocio, “Abarrato” requiere un renderizado dinámico del lado del cliente (Client-Side Rendering mediante la aplicación móvil nativa). Dado que los precios, el inventario de la despensa de cada usuario y la geolocalización cambian constantemente, no es viable servir pantallas estáticas. El servidor entregará únicamente los datos crudos mediante JSON a través de la API REST, y será el dispositivo móvil el que se encargará de construir y actualizar las interfaces visuales en tiempo real, optimizando el consumo de red y disminuyendo la carga de procesamiento en el servidor.

Conclusión:

El desarrollo de la propuesta para “Abarrato” demuestra que se puede abordar un problema económico y logístico con un enfoque centrado en la arquitectura de software bien estructurada. A lo largo del documento, se logró definir una promesa de valor clara, respaldada por la situación económica actual, y traducirla en una experiencia de usuario fluida mediante una arquitectura de información orientada a tareas.
Desde el punto de vista técnico, la estructuración del servidor en tres capas y la elección de nuestro stack tecnológico asegura que el motor del proyecto sea robusto y de fácil escalado. Lo que quedaría pendiente para los siguientes hitos sería la construcción de los recursos en la API REST, el diseño de la base de datos y la conexión final entre el cliente móvil y el backend.

Este primer hito dejó como aprendizaje la importancia del diseño previo a la programación. Entender que la toma de decisiones acerca de la arquitectura son el verdadero cimiento de un proyecto, previniendo problemas de rendimiento, seguridad y usabilidad a futuro.

Bibliografía:

- Instituto Nacional de Estadísticas. (2026). Índice de Precios al Consumidor [Calculadora en línea]. https://www.ine.gob.cl/estadisticas-por-tema/precios-e-inflacion/indice-de-precios-al-consumidor

- Ministerio de Desarrollo Social y Familia. (2026, enero). Valor de la Canasta Básica de Alimentos y Líneas de Pobreza: Informe mensual, enero de 2026 (serie metodología 2024).

- Google Cloud. (s.f.). ¿Qué es PostgreSQL?. https://cloud.google.com/discover/what-is-postgresql?hl=es

- Google Cloud. (s.f.). ¿Qué es una API de REST?. https://cloud.google.com/discover/what-is-rest-api?hl=es

- IBM. (s.f.). ¿Qué es Java Spring Boot?. https://www.ibm.com/think/topics/java-spring-boot

- IronPDF. (s.f.). ¿Para qué se utiliza Node.js?. https://ironpdf.com/es/nodejs/blog/node-help/what-is-nodejs-used-for/?chattype=advance

- Muñoz, C. (2026). Arquitectura de Desarrollo (Móvil y Web) · INGT1003 - Unidad 1: Aspectos Generales de la Arquitectura Web [Apuntes de clase]. Universidad Mayor.

- Piensa Solutions. (s.f.). Mobile First: Qué es y por qué usarlo en diseño responsive. https://www.piensasolutions.com/blog/mobile-first-que-es-y-por-que-usarlo-en-diseno-responsive

- Piensa Solutions. (s.f.). ¿Qué es Flutter y por qué utilizarlo para desarrollar tus aplicaciones?. https://www.piensasolutions.com/blog/que-es-flutter-y-por-que-utilizarlo-para-desarrollar-tus-aplicaciones

Anexos:

Herramientas utilizadas

- Gemini: Se utilizó como herramienta de apoyo estructural para la redacción del documento, organización de las ideas de la propuesta comercial, la generación de ejemplos de redacción y como consultor de errores ortográficos y/o lógicos además de programador para la creación del prototipo de la aplicación.

- Perplexity: Se utilizó como herramienta de motor de búsqueda acelerada para la localización rápida de datos estadísticos gubernamentales e información bibliográfica relevante.

- Licencias:

- OpenJDK (Eclipse Temurin): Distribuido bajo la licencia GNU General Public License v2 with Classpath Exception (GPLv2 + CPE). Permite el uso gratuito, la modificación y la distribución comercial del entorno de ejecución de Java sin obligar a abrir el código fuente de la aplicación propietaria que corre sobre él.

- Java Spring Boot: Desarrollado bajo la Apache License 2.0. Es una licencia de código abierto muy permisiva que otorga al equipo los derechos de uso, modificación y distribución comercial, requiriendo únicamente conservar los avisos de copyright y descargo de responsabilidad originales.

- PostgreSQL: Utiliza la Licencia PostgreSQL. Permite el uso, modificación, distribución y comercialización del motor de base de datos relacional tanto en entornos locales como de producción.

- Google Cloud Platform (GCP): Su utilización para el alojamiento de los servidores y bases de la infraestructura en la nube se rige bajo los Términos de Servicio de Google Cloud y los Acuerdos de Nivel de Servicio (SLA) comerciales correspondientes para la gestión de instancias y servicios administrados.